

Scheduling Functions at Custom Frequencies / 以自定义频率调度函数

`TimedRobot`'s `addPeriodic()` method allows one to run custom methods at a rate faster than the default `TimedRobot` periodic update rate (20 ms). Previously, teams had to make a `Notifier` to run feedback controllers more often than the `TimedRobot` loop period of 20 ms (running `TimedRobot` more often than this is not advised). Now, users can run feedback controllers more often than the main robot loop, but synchronously with the `TimedRobot` periodic functions, eliminating potential thread safety issues.

`TimedRobot` 的 `addPeriodic()` 方法允许以比 `TimedRobot` 默认周期更新 (20 ms) 更快的频率运行自定义方法。以前，队伍必须创建一个 `Notifier`，才能比 `TimedRobot` 的 20 ms 循环更频繁地运行反馈控制器（不建议让 `TimedRobot` 本身运行得比这更频繁）。现在，用户可以比机器人主循环更频繁地运行反馈控制器，同时与 `TimedRobot` 的周期函数同步执行，从而消除潜在的线程安全问题。

The `addPeriodic()` method takes in a `lambda` (the function to run), along with the requested period and an optional offset from the common starting time. The optional third argument is useful for scheduling a function in a different timeslot relative to the other `TimedRobot` periodic methods.

`addPeriodic()` 方法接收一个 `lambda`（要运行的函数）、所需的周期，以及相对于共同起始时刻的可选偏移量。可选的第三个参数可用于将函数安排在与其它 `TimedRobot` 周期方法不同的时间段执行。

Note / 说明

The units for the period and offset are seconds in Java.

在 Java 中，周期和偏移量的单位都是秒。

```
public class Robot extends TimedRobot {
    private Joystick m_joystick = new Joystick(0);
    private Encoder m_encoder = new Encoder(1, 2);
    private Spark m_motor = new Spark(1);
    private PIDController m_controller = new PIDController(1.0, 0.0, 0.5, 0.01);
    public Robot() {
        addPeriodic(() -> {
            m_motor.set(m_controller.calculate(m_encoder.getRate()));
        }, 0.01, 0.005);
    }
    @Override
    public teleopPeriodic() {
        if (m_joystick.getRawButtonPressed(1)) {
            if (m_controller.getSetpoint() == 0.0) {
                m_controller.setSetpoint(30.0);
            } else {
                m_controller.setSetpoint(0.0);
            }
        }
    }
}
```

译注：原示例的 `teleopPeriodic()` 声明缺少返回类型；Java 中应声明为 `public void teleopPeriodic()`。此处按原文保留代码。

The `teleopPeriodic()` method in this example runs every 20 ms, and the controller update is run every 10 ms with an offset of 5 ms from when `teleopPeriodic()` runs so that their timeslots don't conflict (e.g., `teleopPeriodic()` runs at 0 ms, 20 ms, 40 ms, etc.; the controller runs at 5 ms, 15 ms, 25 ms, etc.).

本例中的 `teleopPeriodic()` 方法每 20 ms 运行一次，控制器更新每 10 ms 运行一次，并相对于 `teleopPeriodic()` 的运行时刻偏移 5 ms，使两者的执行时间段不发生冲突（例如，`teleopPeriodic()` 在 0 ms、20 ms、40 ms 等时刻运行；控制器在 5 ms、15 ms、25 ms 等时刻运行）。

Event-Based Programming With EventLoop / 使用 EventLoop 进行事件驱动编程

Many operations in robot code are driven by certain conditions; buttons are one common example. Conditions can be polled with an [imperative programming](#) style by using an if statement in a periodic method. As an alternative, WPILib offers an [event-driven programming](#) style of API in the shape of the EventLoop and BooleanEvent classes.

机器人代码中的许多操作由特定条件驱动；按钮就是一个常见例子。可以采用[命令式编程](#)风格，在周期方法中使用 if 语句轮询条件。另一种选择是，WPILib 通过 EventLoop 和 BooleanEvent 类提供了[事件驱动编程](#)风格的 API。

Note / 说明

The example code here is taken from the EventLoop example project ([Java](#)).

这里的示例代码取自 EventLoop 示例项目 ([Java](#))。

EventLoop / EventLoop

The EventLoop class is a “container” for pairs of conditions and actions, which can be polled using the poll() method. When polled, every condition will be queried and if it returns true the action associated with the condition will be executed.

EventLoop 类是存放条件与动作配对的“容器”，可以使用 poll() 方法对其进行轮询。轮询时，会查询每个条件；如果条件返回 true，就会执行与该条件关联的动作。

```
private final EventLoop m_loop = new EventLoop();
private final Joystick m_joystick = new Joystick(0);
@Override
public void robotPeriodic() {
    // poll all the bindings
    m_loop.poll();
}
```

Warning / 警告

The EventLoop's poll() method should be called consistently in a *Periodic() method. Failure to do this will result in unintended loop behavior.

应在一个 *Periodic() 方法中持续调用 EventLoop 的 poll() 方法。否则，事件循环会出现非预期行为。

BooleanEvent / BooleanEvent

The BooleanEvent class represents a boolean condition: a BooleanSupplier ([Java](#)).

BooleanEvent 类表示一个布尔条件，即一个 BooleanSupplier ([Java](#))。

To bind a callback action to the condition, use ifHigh():

要将回调动作绑定到该条件，请使用 ifHigh():

```
BooleanEvent atTargetVelocity =
    new BooleanEvent(m_loop, m_controller::atSetpoint)
        // debounce for more stability
        .debounce(0.2);

// if we're at the target velocity, kick the ball into the shooter wheel
atTargetVelocity.ifHigh(() -> m_kicker.set(0.7));
```

Remember that button binding is declarative: bindings only need to be declared once, ideally some time during robot initialization. The library handles everything else.

请记住，按钮绑定是声明式的：只需声明一次绑定，最好在机器人初始化期间进行。库会处理其余所有工作。

Composing Conditions / 组合条件

BooleanEvent objects can be composed to create composite conditions. All compositions in Java are done using methods.

可以组合 BooleanEvent 对象来创建复合条件。在 Java 中，所有组合都通过方法完成。

and() / 与运算 and()

The and() composes two BooleanEvent conditions into a third condition that returns true only when both of the conditions return true.

and() 将两个 BooleanEvent 条件组合成第三个条件；只有当两个条件都返回 true 时，这个条件才返回 true。

```
// if the thumb button is held
intakeButton
    // and there is not a ball at the kicker
    .and(isBallAtKicker.negate())
    // activate the intake
    .ifHigh(() -> m_intake.set(0.5));
```

or() / 或运算 or()

The or() composes two BooleanEvent conditions into a third condition that returns true only when either of the conditions return true.

or() 将两个 BooleanEvent 条件组合成第三个条件；只有当至少一个条件返回 true 时，这个条件才返回 true。

```
// if the thumb button is not held
intakeButton
    .negate()
    // or there is a ball in the kicker
    .or(isBallAtKicker)
    // stop the intake
    .ifHigh(m_intake::stopMotor);
```

negate() / 取反 negate()

The negate() composes one BooleanEvent condition into another condition that returns the opposite of what the original conditional did.

negate() 将一个 BooleanEvent 条件组合成另一个条件，其返回值与原条件相反。

```
// and there is not a ball at the kicker
.and(isBallAtKicker.negate())
```

debounce() / 去抖 debounce()

To avoid rapid repeated activation, conditions (especially those originating from digital inputs) can be debounced with the [WPILib Debouncer class](#) using the debounce method:

为了避免快速重复触发，可以使用 debounce 方法，通过 [WPILib Debouncer](#) 类对条件（尤其是来自数字输入的条件）进行去抖：

```
BooleanEvent atTargetVelocity =
    new BooleanEvent(m_loop, m_controller::atSetpoint)
        // debounce for more stability
        .debounce(0.2);
```

rising(), falling() / rising()、falling()

Often times it is desired to bind an action not to the current state of a condition, but instead to when that state changes. For example, binding an action to when a button is newly pressed as opposed to when it is held. This is what the `rising()` and `falling()` decorators do: `rising()` will return a condition that is true only when the original condition returned true in the current polling and false in the previous polling; `falling()` returns a condition that returns true only on a transition from true to false.

通常，希望将动作绑定到条件状态的变化，而不是条件的当前状态。例如，将动作绑定到按钮刚被按下的时刻，而不是按钮被持续按住的期间。这正是 `rising()` 和 `falling()` 装饰方法的作用：`rising()` 返回的条件，只有在原条件于当前轮询中返回 true、且于上一次轮询中返回 false 时才为 true；`falling()` 返回的条件，只有在从 true 变为 false 时才返回 true。

Warning / 警告

Due to the “memory” these conditions have, do not use the same instance in multiple places.

由于这些条件具有“记忆”，不要在多个位置使用同一个实例。

```
// when we stop being at the target velocity, it means the ball was shot
atTargetVelocity
    .falling()
    // so stop the kicker
    .ifHigh(m_kicker::stopMotor);
```

Downcasting BooleanEvent Objects / 对 BooleanEvent 对象进行向下转型

To convert `BooleanEvent` objects to other types, most commonly the `Trigger` subclass used for [binding commands to conditions](#), the generic `castTo()` decorator exists:

要将 `BooleanEvent` 对象转换为其他类型（最常见的是用于将 [Command 绑定到条件](#) 的 `Trigger` 子类），可以使用泛型装饰方法 `castTo()`：

```
Trigger trigger = booleanEvent.castTo(Trigger::new);
```

Note / 说明

In Java, the parameter expects a method reference to a constructor accepting an `EventLoop` instance and a `BooleanSupplier`.

在 Java 中，该参数需要一个指向构造函数的方法引用；该构造函数应接收一个 `EventLoop` 实例和一个 `BooleanSupplier`。